# Thi online, chấm trên máy BTC

Hướng dẫn mô hình website cloud + bộ chấm tại máy BTC

Website: oan-tu-ti-ai-arena.vercel.app/competition

Không cần mua tên miền, mở cổng máy chấm hay dùng tunnel. Website nhận bài trên Vercel; máy BTC chủ động lấy công việc và gửi kết quả qua HTTPS.

| Bên tham gia | Cần dùng gì? |
|---|---|
| Sinh viên | Trình duyệt; Python và editor để tự phát triển bot nếu muốn. |
| BTC điều hành | Trình duyệt và tài khoản được cấp quyền Ban tổ chức. |
| Người kỹ thuật | Vercel + Neon; máy chấm có Docker Desktop. Git và Node 22.16+ phục vụ thiết lập. |

## Luồng hoạt động

| Thành phần | Trách nhiệm |
|---|---|
| Sinh viên → Vercel | Đăng nhập, đổi mã đội, nộp bot.py, xem bảng đấu và replay. |
| Vercel → Neon | Lưu tài khoản, source, phiên bản luật, hàng đợi và kết quả. |
| Docker → Vercel | Xin việc, giữ lease/heartbeat, gửi kết quả bằng secret riêng. |
| Docker → sandbox | Mỗi bot chạy riêng trong CPython WASI/Wasmtime; không được cấp mạng, secret hoặc tệp của máy BTC. |

Vercel không chạy Python của bài thi. Worker không có mật khẩu database. Chỉ người kỹ thuật giữ quyền database và cấu hình triển khai.

# Chuẩn bị website và máy chấm

Cấu hình một lần · người kỹ thuật

## 1. Website

Kết nối repository với Vercel; dùng vercel.json đã có. Kết nối Neon và đặt DATABASE_URL trong môi trường Production. Build tạo schema arena_competition riêng, giữ dữ liệu Vercel hiện có.

## 2. Tạo cấu hình worker trên máy BTC

```
npm run judge:cloud:init
```

Lệnh tạo .env.judge.local và .secrets/vercel-judge-token. Đặt đúng nội dung token làm biến server JUDGE_TOKEN trong Vercel Production, sau đó redeploy. Hai file local này không nằm trong Git hoặc gói upload Vercel.

**Không** đưa token vào NEXT_PUBLIC_*, bài nộp hoặc nhóm sinh viên. Người quản lý có thể nhập token trong phần Environment Variables của Vercel; lệnh PowerShell nằm trong hướng dẫn Markdown đi kèm.

## 3. Cấp quyền BTC

BTC đăng ký tài khoản thật trên website. Chủ database xác nhận đúng tài khoản rồi cấp quyền:

```
node --env-file=.env.vercel.local scripts/vercel-admin.mjs
  your@email.com
```

Lệnh trên trình bày trên hai dòng cho dễ đọc; khi chạy, nhập thành một dòng. Sau đó vào /competition → Ban tổ chức. Không cấp quyền dựa trên tên hiển thị hoặc email tự khai.

## 4. Bật Docker

```
docker compose -f compose.judge.yaml --env-file
  .env.judge.local up -d --build
```

Nhập lệnh thành một dòng. Docker Desktop phải đang chạy Linux containers. Worker giới hạn 2 CPU, 2 GiB RAM; nên bắt đầu với máy có RAM 8 GiB trở lên và SSD. Đây chưa phải cấu hình được chứng nhận cho 4.000 người.

Mã nguồn: github.com/nguyenlean17/oan-tu-ti-ai-arena

# Từ nộp bài đến công bố

Thao tác thường ngày · BTC và sinh viên

## Ban tổ chức

1. Vào Ban tổ chức → Tạo giải đấu mới. Chọn số bảng, suất đi tiếp, hạn nộp và giới hạn thời gian/bộ nhớ. Chốt luật trước khi nhận bài.

2. Nhập danh sách đội và sinh viên; phát mã tham gia riêng cho đúng người. Mã được gắn với email và MSSV trong danh sách BTC.

3. Mở nhận bài. Theo dõi bộ chấm và hàng đợi. Bài lỗi bị đánh dấu không hợp lệ; sinh viên có thể sửa và nộp lại trong hạn.

4. Hết hạn, khóa danh sách sau khi kiểm tra hết bài chờ. Hệ thống lấy bài hợp lệ mới nhất trước hạn, chia bảng và chạy các cặp đổi bên.

5. Kết thúc loại trực tiếp, kiểm tra replay và công bố. Xuất toàn bộ giải để lưu source đã khóa, config và diễn biến.

## Sinh viên

Mở website → đăng ký/đăng nhập → nhập mã để vào đội → nộp bot.py → xem trạng thái kiểm tra bài → theo dõi lịch đấu, bảng điểm và replay.

Có thể nhập bot.py từ commit GitHub đầy đủ trong repo công khai đã được BTC gắn cho đội. Git không thay thế tài khoản, mã đội hoặc sandbox. Không cần cài Docker để tham gia.

## Giới hạn và thời gian chờ

Mỗi đội: một phiên bản mỗi 5 giây, tối đa hai bài chờ kiểm tra và 200 phiên bản trong giải. Python WASI không hỗ trợ pip, socket, subprocess hay extension native.

Worker rảnh lấy việc theo khoảng tăng dần khoảng 5–15 giây. Trang đang mở cập nhật khoảng 15 giây; giải đã công bố ngừng polling. Thời gian trả kết quả còn phụ thuộc hàng đợi, CPU và bot; không cam kết tức thì.

Trước ngày thi, thử bot hợp lệ, bot sai luật, chấm trận và replay bằng đúng image/runtime sẽ dùng trong giải.

# Vận hành an toàn, tiết kiệm

Sự cố, hạn mức và bảo toàn dữ liệu

## Mất mạng hoặc máy chấm tắt

Website vẫn nhận bài; công việc nằm trong PostgreSQL. Khi worker trở lại, nó tiếp tục nhận việc. Lease 90 giây, heartbeat 20 giây; kết quả lease cũ bị từ chối. Gửi lại cùng kết quả không cộng điểm hai lần.

Lỗi hạ tầng được thử hữu hạn. Nếu job thất bại, BTC xem hàng đợi, xử lý nguyên nhân rồi thử lại. Nếu runtime khác phiên bản giải đã khóa, hệ thống dừng chấm để tránh âm thầm đổi luật.

## Hết buổi: dừng worker

```
docker compose -f compose.judge.yaml --env-file
  .env.judge.local stop
```

Nhập thành một dòng. Lần sau bật lại bằng up -d. Polling liên tục có thể giữ database luôn thức và tiêu hao hạn mức miễn phí; hãy dừng worker khi không tổ chức giải.

## Sao lưu và ranh giới dữ liệu

Xuất giải từ admin panel để giữ source đã khóa, cấu hình và replay. Dùng backup/export PostgreSQL của tài khoản Neon để bảo vệ toàn bộ database; export giải không thay thế backup tài khoản.

Dữ liệu trong volume Docker cũ vẫn được giữ tại máy và không tự nhập lên website công khai. Dữ liệu Vercel trước đây được giữ. File .env.vercel.local có quyền database: chỉ người kỹ thuật giữ, không gửi cho máy chấm khác.

## Giới hạn hiện tại

Cấu hình pilot tuần tự hóa thao tác ghi bằng khóa giao dịch PostgreSQL; truy vấn đọc không chờ khóa ghi đó. Khi đo thấy nghẽn, cần tách khóa theo giải/job và cache bảng điểm. Chưa cam kết chịu tải 4.000 sinh viên đồng thời hoặc chi phí luôn bằng 0.

Chưa có SSO trường, xác minh email tự động hoặc khôi phục mật khẩu. BTC xác nhận danh sách, phát mã riêng; duy trì cập nhật OS/runtime và chỉ dùng máy chấm do BTC quản lý.

Hướng dẫn lệnh đầy đủ và kiểm thử: deploy/VERCEL_JUDGE.md trong repository.